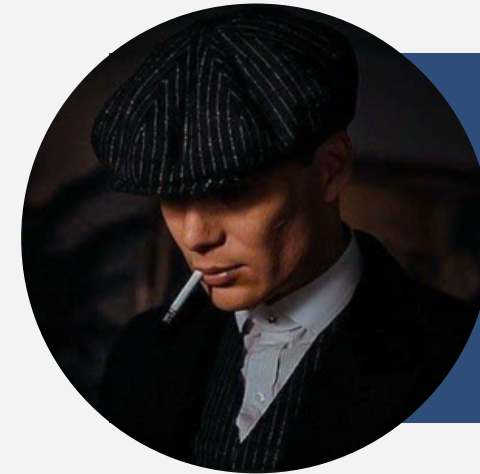


Presentation

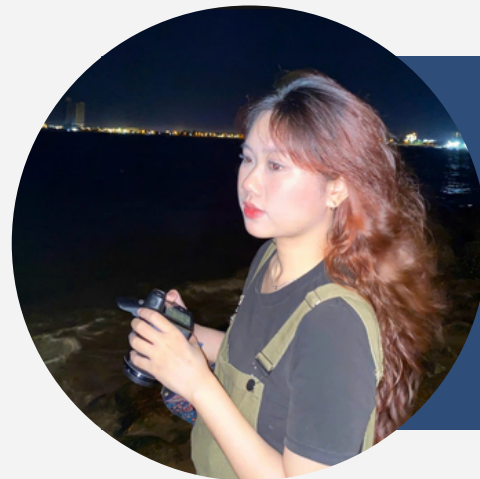
XSS DETECTOR — AUTOMATED XSS PAYLOAD CLASSIFICATION USING NLP

WEB SECURITY (IAW301)

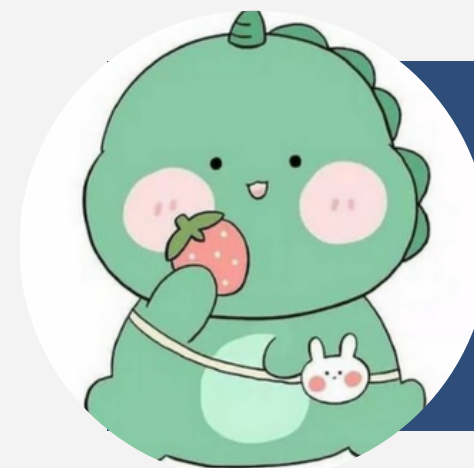
17 March, 2026



Phạm Hoàng Hải Nguyễn
SE180511

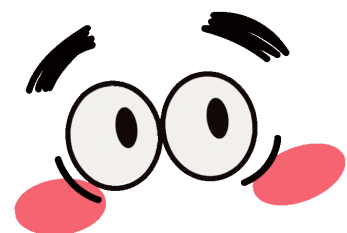


Nguyễn Trúc Quỳnh
SE180280



Giả Thị Nhã Trân
Se180405

MEMBERS



AGENDA OVERVIEW

01

INTRODUCTION

02

THEORETICAL BACKGROUND

03

DATASET & PREPROCESSING

04

MODEL DESIGN

05

EXPERIMENTAL RESULTS

06

MODEL COMPARISON

07

SYSTEM IMPLEMENTATION

08

DISCUSSION

09

CONCLUSION & FUTURE WORK

INTRODUCTION

- Web applications are essential to modern digital infrastructure
- They are constantly targeted by cyber attacks
- One of the most dangerous vulnerabilities is Cross-Site Scripting (XSS)
- Attackers inject malicious scripts into trusted websites
- Scripts execute silently inside the victim's browser

Consequences:

- Session hijacking
- Cookie theft
- Credential stealing
- Redirection to malicious sites



RESEARCH MOTIVATION

OVERVIEW

- XSS appears in the OWASP Top 10 every year
- Represents 20%+ of all reported web vulnerabilities

PROPOSNENTS



TRADITIONAL DETECTION METHODS:

- Rule-based filtering
- Signature-based detection
- Regular expressions (Regex)



PROBLEM:

These methods are easily bypassed by:

- Payload encoding
- Case variation (ScRiPt)
- Alternative event handlers (onerror, onmouseover)



RESEARCH OBJECTIVES

RESEARCH OBJECTIVES

This project aims to:

1. Build an automated XSS payload classification system
2. Compare three different models:
 - SVM + TF-IDF (classical ML)
 - DistilBERT (English transformer)
 - PhoBERT (Vietnamese transformer)
3. Evaluate performance using standard metrics:
Accuracy | Precision | Recall | F1-Score
4. Deploy a real-time detection system via Flask REST API



RESEARCH SCOPE

TASK: BINARY CLASSIFICATION

- Label 1 → XSS payload
- Label 0 → Benign text

DATASET:

- 500 raw samples collected
- 98 duplicates removed
- 402 final samples used

500

Raw Samples

98

Duplicates Removed

402

Final Samples

61.2%

XSS Payloads

38.8%

Benign Text

PROJECT WORKFLOW



Dataset Collection



Data Cleaning



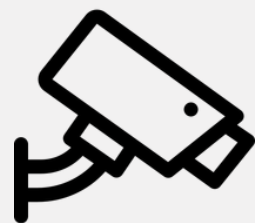
Text Preprocessing



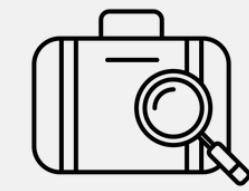
Feature Extraction



Model Training



Model Evaluation



Deployment via Flask API



WHAT IS XSS?

Cross-Site Scripting (XSS) is a vulnerability that occurs when:

- User input is inserted into HTML output without validation or encoding.
- As a result: malicious JavaScript executes in the victim's browser.

Classic example payload:

```
<script>alert(document.cookie)</script>
```

When successfully injected:

- The script runs in the victim's browser
- Attacker receives the cookie value → active session stolen
- No visible sign to the victim



XSS ATTACK PROCESS

01

ATTACKER CRAFTS A MALICIOUS PAYLOAD

02

PAYLOAD INJECTED INTO THE WEB APPLICATION (FORM, URL, HEADER)

03

VICTIM LOADS THE INFECTED PAGE

04

BROWSER EXECUTES THE MALICIOUS SCRIPT

05

ATTACKER COLLECTS STOLEN DATA (COOKIES, CREDENTIALS, KEYSTROKES)



Reflected XSS (Non-Persistent)

- Payload returned immediately in HTTP response
- Delivered via crafted URL in email or message
- Most common type – easiest to exploit

Stored XSS (Persistent)

- Payload permanently stored in server database
- Executes for every user visiting the page
- Most dangerous – no URL trick needed

DOM-Based XSS (Client-side)

- Attack occurs entirely in the browser DOM
- Payload never reaches the server
- Server-side filters are completely blind to it

TYPES OF XSS



Blind XSS

- Malicious payload executes in administrator panel or backend system
- Attacker cannot see the result directly
- Detection usually relies on out-of-band channels (e.g., external callbacks)

Self-XSS

- Payload executed by the attacker themselves in their own browser
- Often used in social engineering attacks
- Victims are tricked into copying and executing malicious code

Mutation XSS (mXSS)

- Exploits browser HTML parsing behavior
- Input that appears safe becomes malicious after browser DOM re-interpretation
- Difficult to detect using traditional filtering mechanisms

OTHER XSS VARIANTS

TRADITIONAL XSS DETECTION ■

Current WAF Tools:

- ModSecurity
- OWASP Core Rule Set (CRS)
- Hundreds of regex patterns
- Fast and well-tested

Bypass Techniques:

- Encoding: `one...`
 - Case: `ScRiPt` instead of `script`
 - Alt events: `onerror`, `onmouseover`
 - Protocol: `javascript:alert(1)`
 - Double enc: `%253Cscript%253E`
- Cannot detect zero-day payloads





DATASET & PREPROCESSING

DATASET WAS COLLECTED FROM PUBLIC CYBERSECURITY SOURCES SUCH AS:

- OWASP XSS Filter Evasion Cheat Sheet
- PortSwigger Web Security Academy
- Security repositories on GitHub

DATASET STATISTICS:

- Raw samples: 500
- After cleaning: 402 samples

500

Raw Samples

98

Duplicates Removed

402

Final Dataset

61.2%

XSS Payloads



DATASET & PREPROCESSING

CLASS DISTRIBUTION:

- XSS payloads
- Benign input

PREPROCESSING STEPS:

- Lowercase normalization
- Removing duplicates
- Token cleaning while preserving HTML structures

Split	Samples	Ratio	Purpose
Train	280	69.7%	Model training
Validation	61	15.2%	Hyperparameter tuning, early stopping
Test	61	15.2%	Final evaluation (used exactly once)

FEATURE EXTRACTION — TF-IDF CHARACTER N-GRAMS



FEATURE:

- Text payloads must be converted into numerical features before training machine learning models.
- We use TF-IDF (Term Frequency – Inverse Document Frequency).
- TF-IDF measures the importance of words within a document relative to the entire dataset.

KEY CONFIGURATION:

- Character-level n-grams
- n-gram range: 2 – 5
- Maximum features: 10,000

Parameter	Value	Rationale
analyzer	char	Character-level analysis instead of word-level
ngram_range	(2, 5)	Captures sequences from 2 to 5 characters
max_features	10,000	Limits vocabulary size
sublinear_tf	True	Uses $\log(1+tf)$ instead of raw tf
strip_accents	unicode	Normalizes Unicode diacritics

MODEL 1: SVM + TF-IDF — RESULTS ■

MODEL 1 USES:

- TF-IDF + Support Vector Machine (SVM)
- SVM attempts to find an optimal decision boundary that separates XSS payloads from benign input.

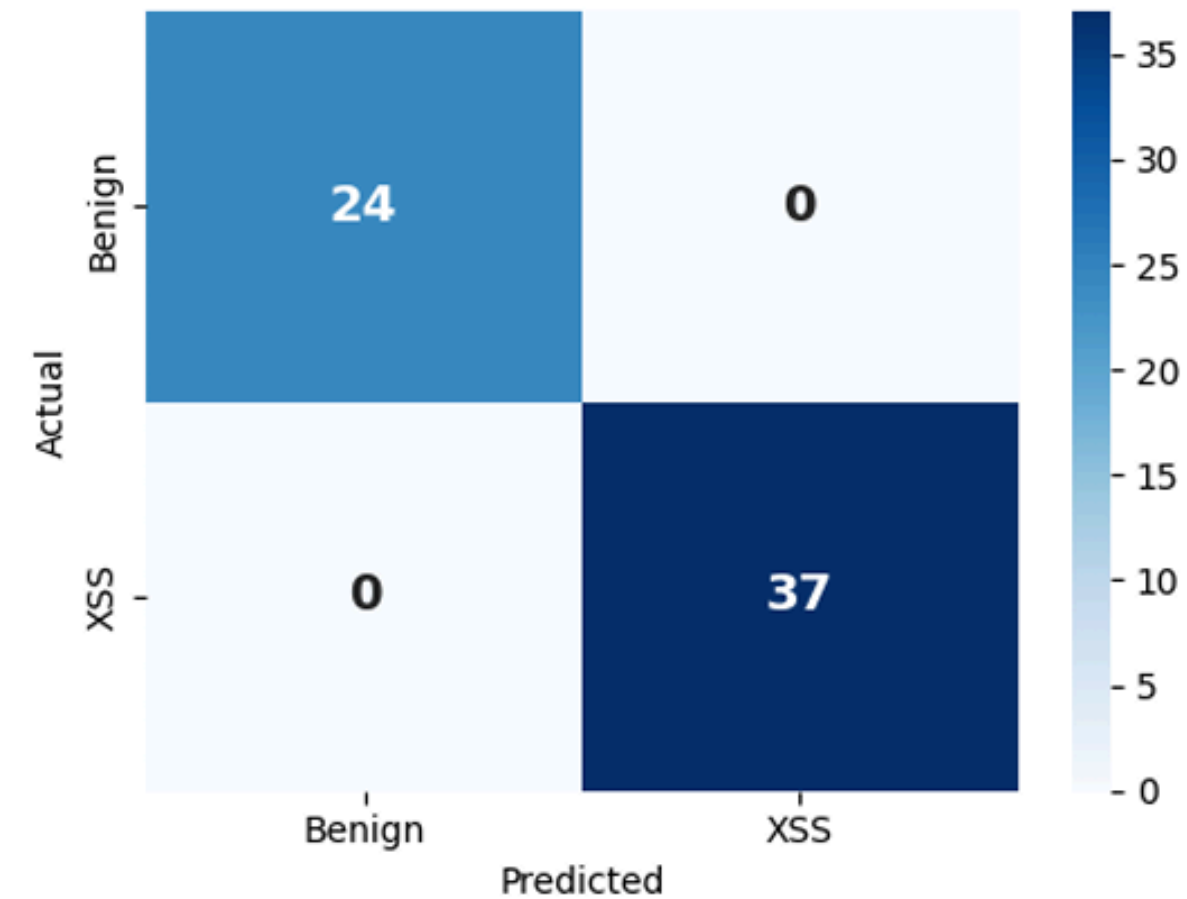
TRAINING CONFIGURATION:

- Kernel: RBF
- Regularization parameter: $C = 1$

RESULTS:

- Accuracy: 100%
- Precision: 1.00
- Recall: 1.00
- F1 Score: 1.00

Confusion Matrix — SVM + TF-IDF (Test Set)



Metric	Val Set	Test Set	Val-Test Gap
Accuracy	100.00%	100.00%	0.00%
Precision (XSS)	1.0000	1.0000	0.00%
Recall (XSS)	1.0000	1.0000	0.00%
F1-Score (XSS)	1.0000	1.0000	0.00%
Training Time	—	1.71 seconds	—

MODEL 2: DISTILBERT — ARCHITECTURE & RESULTS

DistilBERT is a lightweight transformer model derived from BERT.

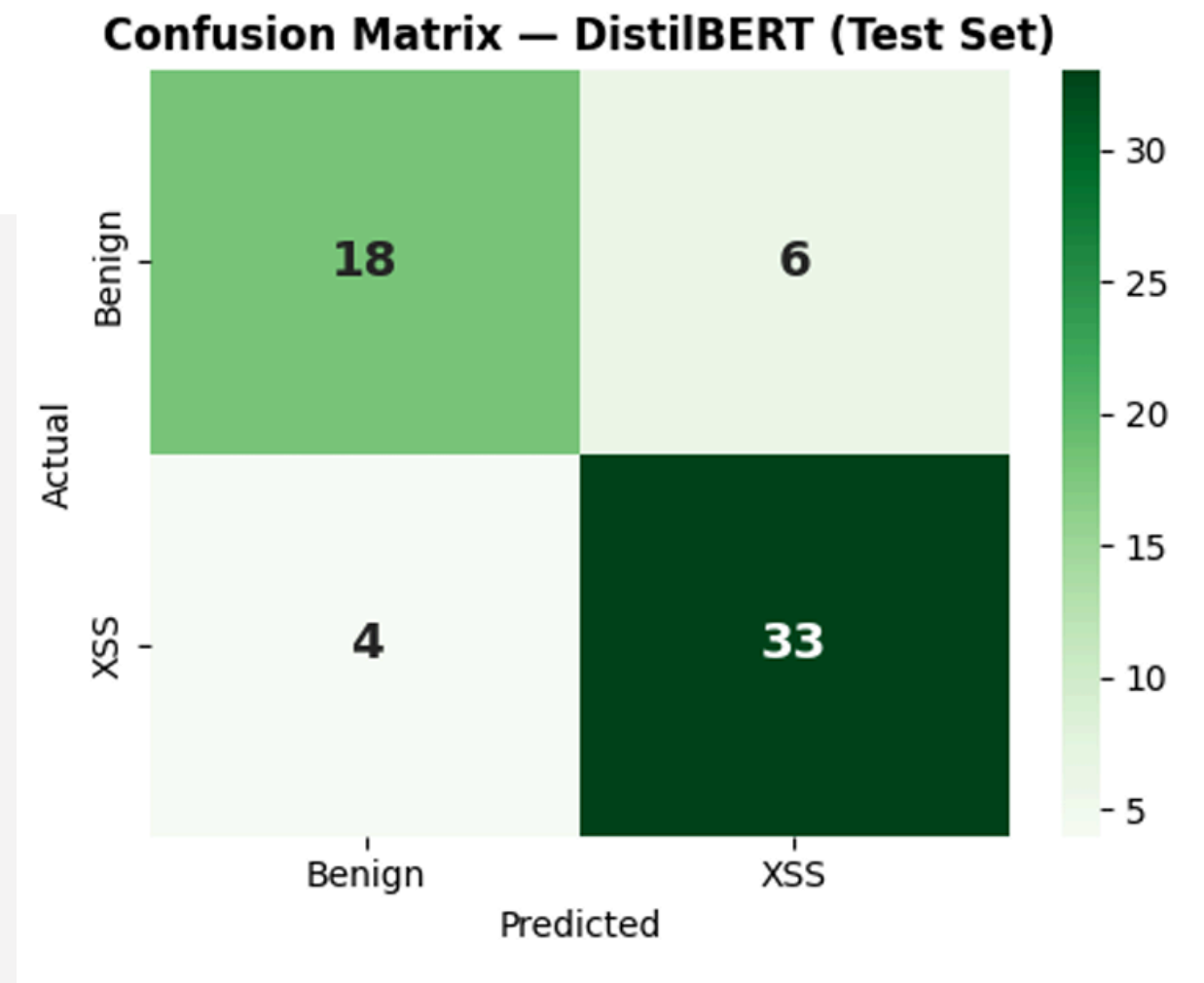
CHARACTERISTICS:

- 6 transformer layers
- 66 million parameters
- Faster training compared to BERT

TRAINING CONFIGURATION:

- Epochs: 5
- Batch size: 16
- Max sequence length: 128

RESULTS ACCURACY: 83.61%



Class	Precision	Recall	F1-Score	Support
Benign (0)	0.8182	0.7500	0.7826	24
XSS (1)	0.8462	0.8919	0.8684	37
Weighted Avg	0.8355	0.8361	0.8346	61
Accuracy	—	—	0.8361	61

MODEL 3: PHOBERT — ARCHITECTURE & RESULTS

PhoBERT is a transformer model designed for Vietnamese language processing.

ARCHITECTURE:

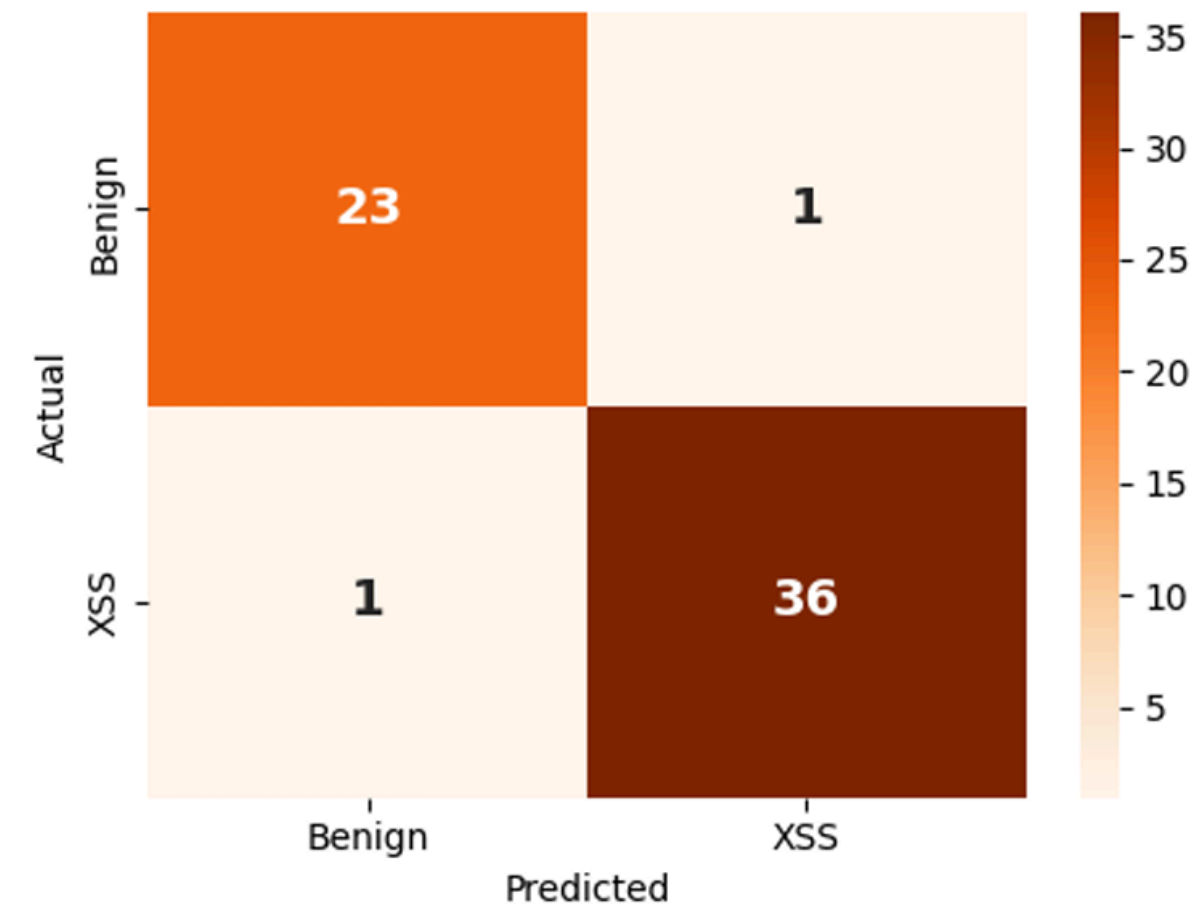
- 12 transformer layers
- 135 million parameters
- Byte-Pair Encoding tokenizer

TRAINING CONFIGURATION:

- Epochs: 5
- Batch size: 8

RESULTS ACCURACY: 96.72%

Confusion Matrix — PhoBERT (Test Set)



Class	Precision	Recall	F1-Score	Support
Benign (0)	0.9583	0.9583	0.9583	24
XSS (1)	0.9730	0.9730	0.9730	37
Weighted Avg	0.9672	0.9672	0.9672	61
Accuracy	—	—	0.9672	61

MODEL COMPARISON

Model	Accuracy	Precision	Recall	F1-Score	Train Time
SVM + TF-IDF	100.00%	1.0000	1.0000	1.0000	1.71s
DistilBERT	83.61%	0.8462	0.8919	0.8684	~17 min
PhoBERT	96.72%	0.9730	0.9730	0.9730	~78 min

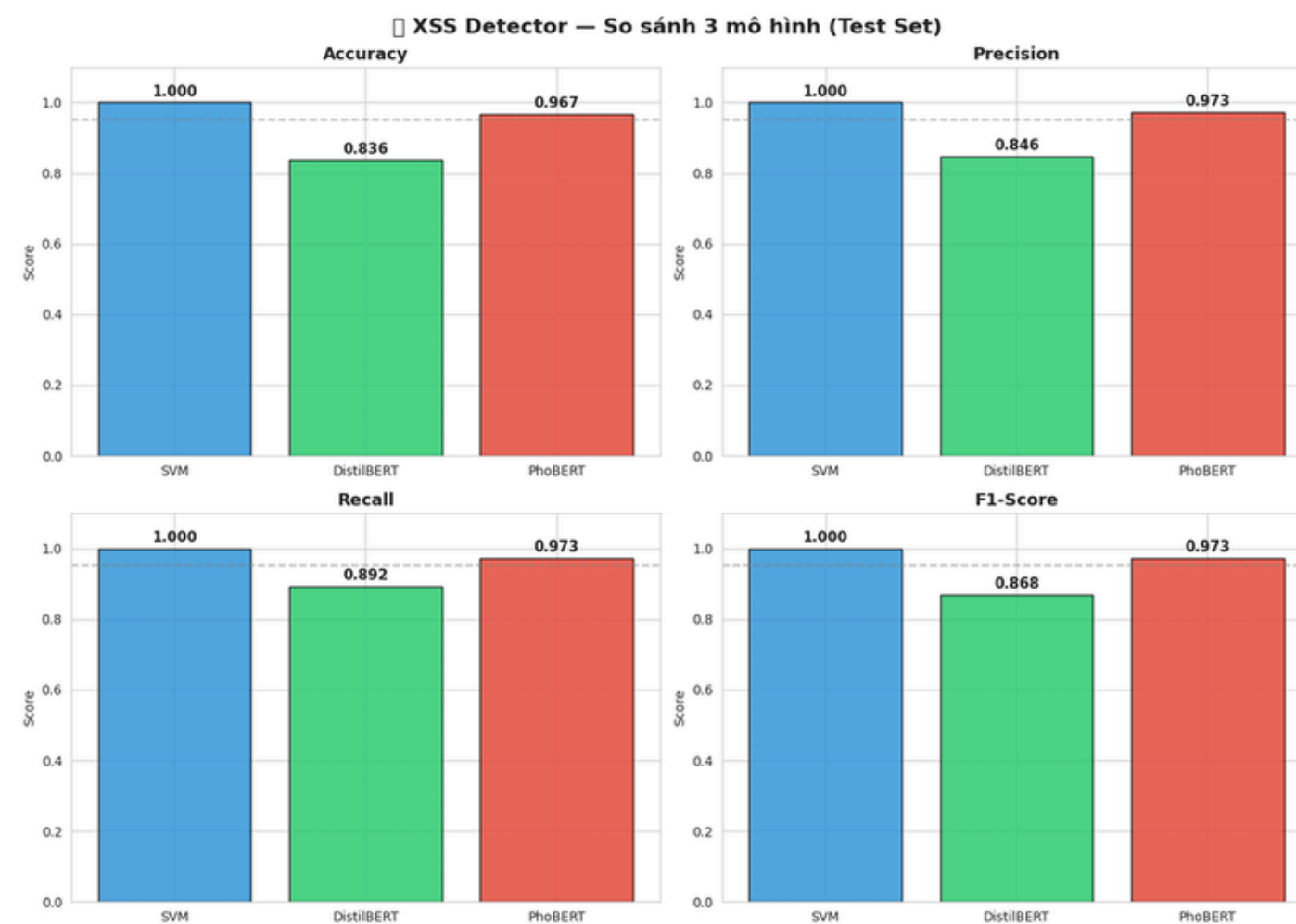
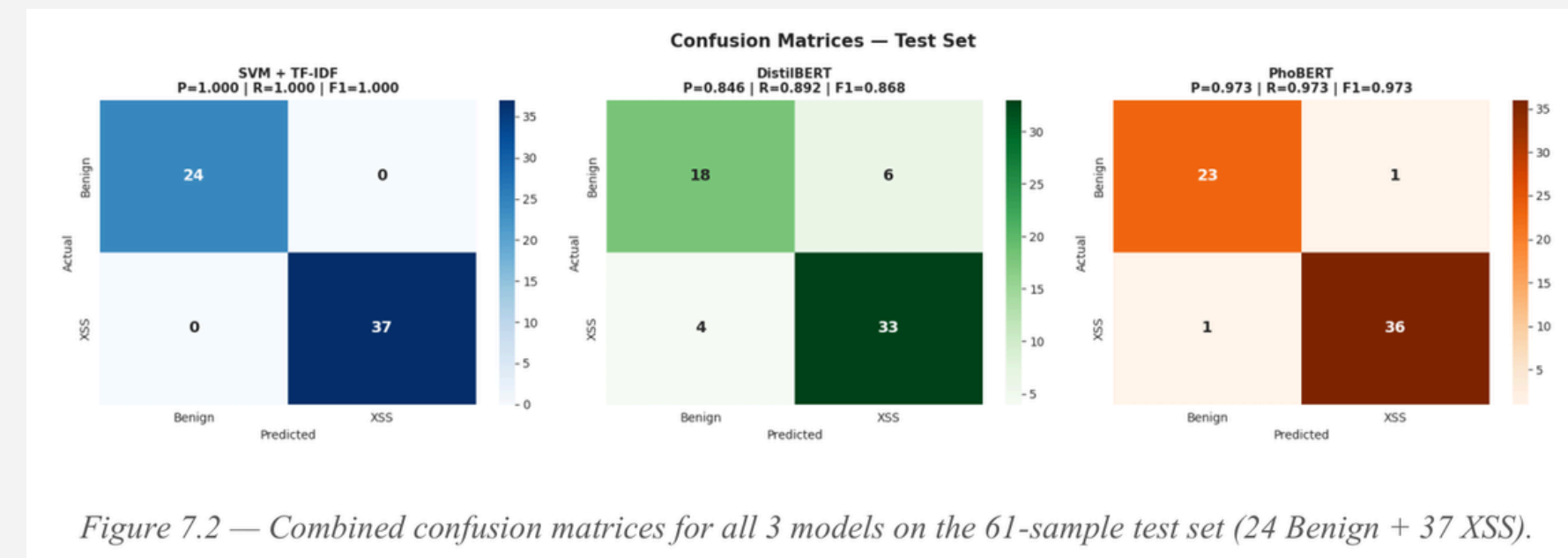


Figure 7.1 — Comparison chart of 4 evaluation metrics: SVM + TF-IDF vs DistilBERT vs PhoBERT on the test set.



CONFUSION MATRIX COMPARISON

— ALL 3 MODELS

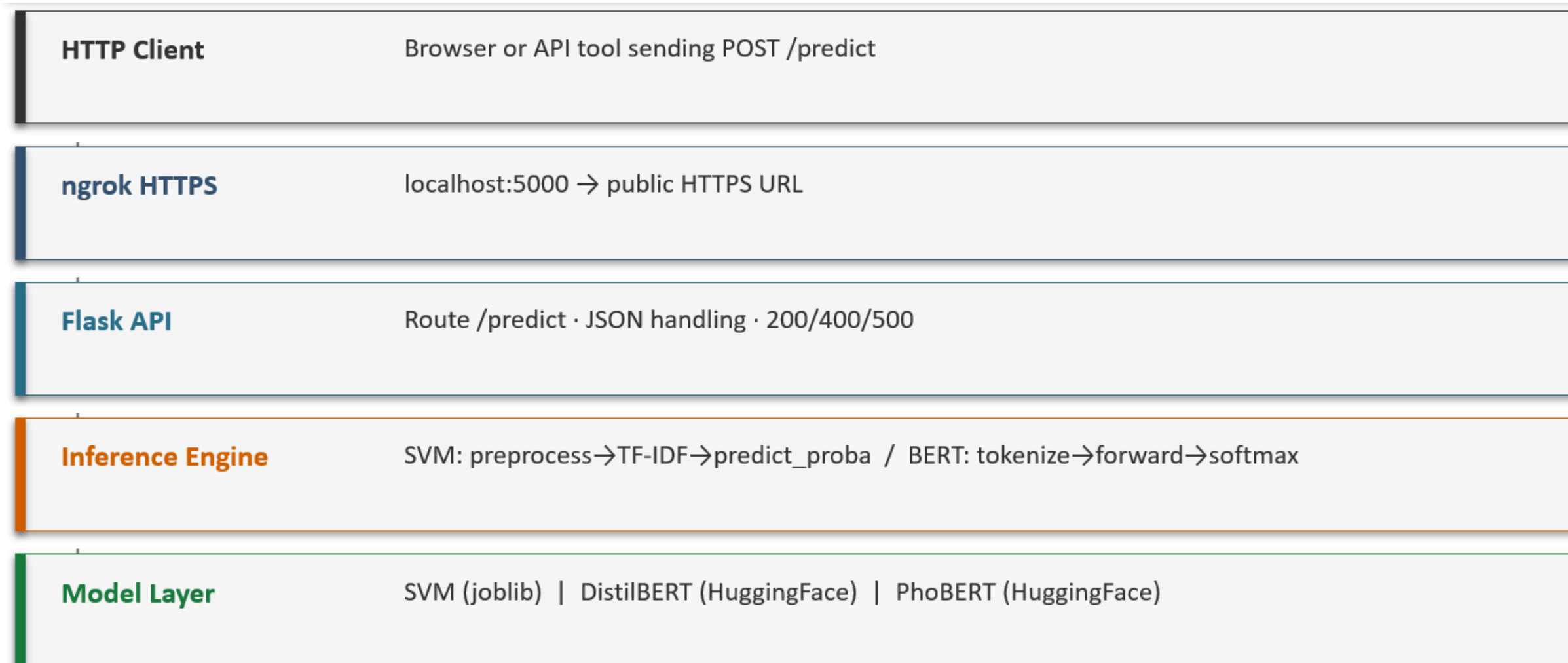
Key observations:

- SVM correctly classifies nearly all samples
- PhoBERT shows strong performance with minimal misclassification
- DistilBERT shows slightly lower recall for benign samples



Figure 7.2 — Combined confusion matrices for all 3 models on the 61-sample test set (24 Benign + 37 XSS).

SYSTEM ARCHITECTURE & FLASK API



Input:

- XSS payload string

Output:

- Prediction result (XSS or benign)

API Request: POST /predict → {"payload": "<script>...", "model": "svm"} → {"label": "XSS", "confidence": 0.987}

Key findings:

- Machine learning methods can effectively detect XSS payloads.
- Traditional models like SVM combined with TF-IDF can outperform deep learning models when the dataset size is small.
- Transformer models require larger datasets to achieve optimal performance.
- The deployed system allows real-time testing of suspicious payloads.

DISCUSSION — KEY FINDINGS

The SIMPLEST model (SVM) achieves the BEST result — 100% accuracy in 1.71 seconds.

Why SVM accuracy 100%

Char n-grams capture XSS syntax perfectly. Benign has zero JS → clear separation. Small dataset (280) favors SVM over deep learning. 10K-dim sparse space = SVM strength.

DistilBERT 83.61%

English pretraining ≠ XSS code. Tokens map to [UNK]. Class imbalance amplifies FP bias. 280 samples insufficient for Transformer.

PhoBERT 96.72%

BPE 64K vocab → fewer [UNK] on code chars. 12 layers vs 6. Vietnamese diacritics training → robust to special characters.

LIMITATIONS & FUTURE WORK

Limitations:

- Dataset size relatively small
- Limited variety of payloads
- Possible adversarial bypass techniques

Future Work:

- Expand dataset to thousands of samples
- Integrate additional attack detection (SQL injection)
- Implement explainable AI methods
- Deploy system for real-time web traffic monitoring

CONCLUSION

Contributions:

- Comparative analysis of three ML models
 - Implementation of an automated detection system
 - Deployment of a Flask-based API for real-time prediction
- The results show that SVM with TF-IDF achieves the best performance for this dataset.

This project demonstrates that ML can effectively detect XSS payloads.
SVM+TF-IDF is the strongest baseline. Transformer models show potential with larger datasets.

SVM + TF-IDF — 100% accuracy, 1.71s

Character n-gram TF-IDF naturally matches XSS syntax structure. Best for production with low latency. Zero errors on test set.

PhoBERT — 96.72% accuracy, ~78 min

Impressive 2nd place despite Vietnamese pretraining. BPE tokenizer handles code chars better than WordPiece. Best Transformer option.

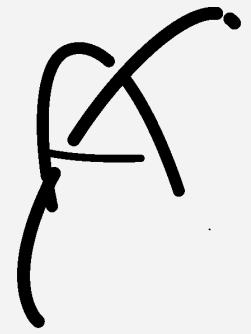
DistilBERT — 83.61% accuracy, ~17 min

Weakest: English pretraining ≠ XSS code. High FP rate (25% Benign misclassified). Needs domain-specific pretraining to improve.

REFERENCES



- [1] OWASP Foundation (2021). OWASP Top 10. owasp.org/Top10
- [2] OWASP. XSS Filter Evasion Cheat Sheet. cheatsheetseries.owasp.org
- [3] PortSwigger. Web Security Academy – XSS. portswigger.net
- [4] Devlin et al. (2018). BERT. [arXiv:1810.04805](https://arxiv.org/abs/1810.04805)
- [5] Sanh et al. (2019). DistilBERT. [arXiv:1910.01108](https://arxiv.org/abs/1910.01108)
- [6] Nguyen & Nguyen (2020). PhoBERT. EMNLP 2020. [arXiv:2003.00744](https://arxiv.org/abs/2003.00744)
- [7] Liu et al. (2019). RoBERTa. [arXiv:1907.11692](https://arxiv.org/abs/1907.11692)
- [9] Cortes & Vapnik (1995). Support-vector networks. Machine Learning 20(3)
- [12] swisskyrepo. PayloadsAllTheThings. github.com/swisskyrepo
- [15] Scikit-learn developers (2023). scikit-learn. JMLR 12, 2825–2830



Thank you

17 March, 2026

